DasIoT Backend Technical Interview

Coding and System Design Exercises

Candidate Instructions

- Use either TypeScript or Python unless otherwise agreed with the interviewer.
- You may use the standard library. Clarifying questions are encouraged.
- Please explain your assumptions, trade-offs, and expected time/space complexity.
- Focus on correctness, readability, maintainability, and testability.

Exercise 1 - In-Memory Dictionary

Build an in-memory dictionary that supports progressively more flexible search operations. Do not assume a specific data structure; choose one that fits the requirements and explain your decision.

Part A - Exact Match

Implement the following operations:

```
setup(words)            # Load or replace the dictionary contents
contains(word) -> bool   # Return true only for a complete-word match
```

Example:

```
setup(["cat", "car", "bar"])
contains("cat")  -> true
contains("ca")   -> false
contains("bat")  -> false
```

Assumptions:

- Words contain lowercase English letters a-z.
- Duplicate words do not change the result.
- Calling setup again replaces the previous dictionary contents.

Part B - Prefix Search

Add a prefix lookup operation:

```
startsWith(prefix) -> bool

startsWith("ca") -> true
startsWith("ba") -> true
startsWith("cr") -> false
```

Part C - Wildcard Search

Extend the dictionary with pattern matching:

- '?' matches exactly one character.
- '*' matches zero or more characters.
- The pattern must match the entire word.
- A pattern may contain more than one wildcard.

```
search("c?t")  -> true
search("*at")  -> true
search("ca*")  -> true
search("cr*")  -> false
search("*")    -> true  # when the dictionary is not empty
```

Exercise 2 - Distributed Document Search Platform

Design a backend platform that allows users to upload, view, and search text documents or books. You are not expected to write implementation code for this exercise; focus on architecture, data flow, trade-offs, and failure handling.

Functional Requirements

1. Users can upload text files or books.
2. Users can view or download an uploaded document.
3. Users can search for keywords across documents they are authorized to access.
4. Search results include the document, matching passages, and highlighted keywords.
5. Users can delete a document.

Scale and Performance Assumptions

Metric	Assumption
Users	1 million
Documents	10 million
Average document size	2 MB
Peak upload rate	100 documents/second
Peak search traffic	3,000 requests/second
Search latency target	Normally under 500 ms
Indexing freshness	Searchable within 5 minutes

Reliability and Security Requirements

- Document processing may be asynchronous and may be retried.
- Retries must not create duplicate documents or duplicate search indexes.
- Users must never see documents they are not authorized to access.
- The system should tolerate individual service or worker failures.
- Document deletion must eventually remove both stored content and searchable data.

Topics to Cover

- API design and core data model
- Document storage and metadata storage
- Upload and asynchronous processing pipeline
- Message queue, retries, idempotency, and dead-letter handling
- Search indexing and consistency between the database and search engine
- Authorization and permission filtering
- Scaling application services, workers, and search infrastructure
- Deployment considerations for containers and Kubernetes
- Monitoring, logs, metrics, tracing, and incident investigation

Discussion Scenarios

1. A user uploads a document successfully, but it is still not searchable after 30 minutes.
2. A worker crashes while processing a document.
3. The same queue message is delivered more than once.
4. A user deletes a document while it is still being indexed.
5. Search traffic suddenly increases by ten times.

The interviewer may adjust or extend the requirements during the discussion. Clearly state assumptions and explain why you chose each major component.